Problemas propuestos 1.1

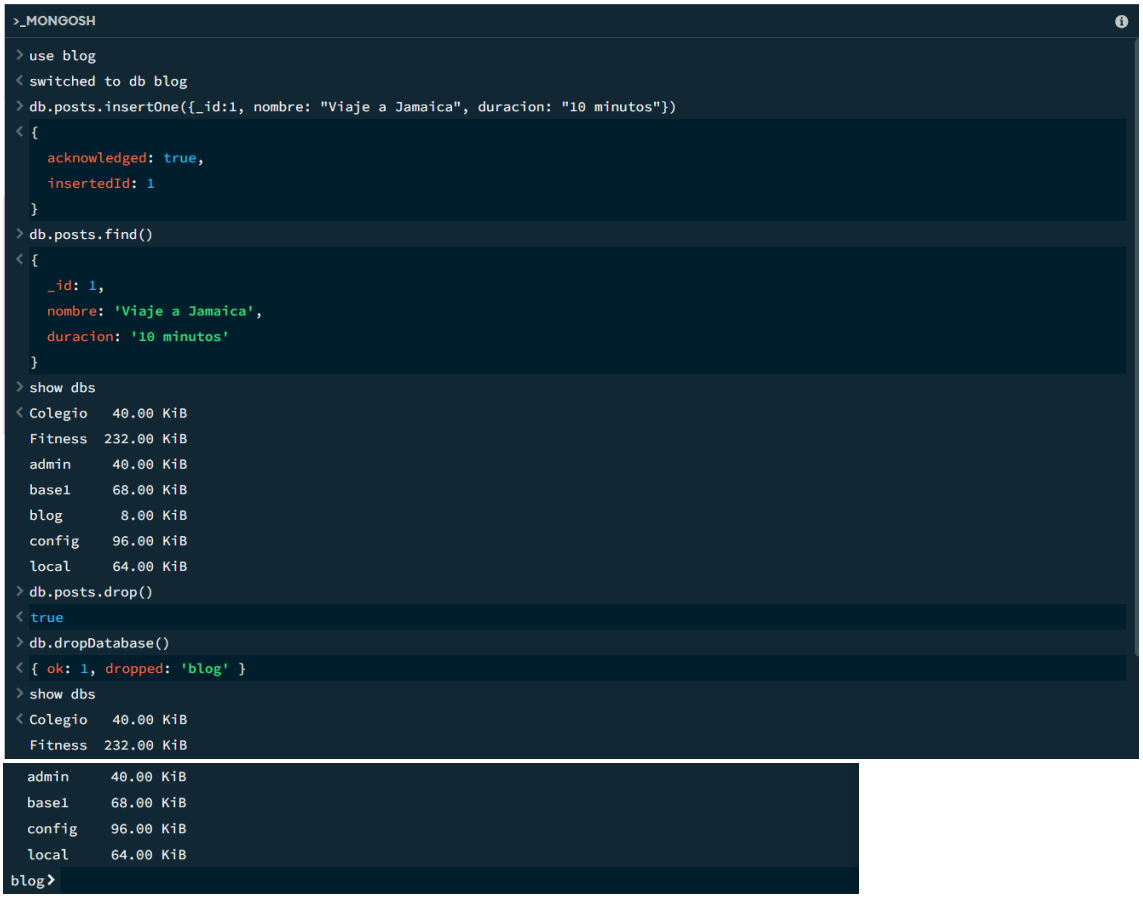
1. Insertar 2 documentos en la colección clientes con '_id' no repetidos
2. Intentar insertar otro documento con clave repetida.
3. Mostrar todos los documentos de la colección libros.





Problemas propuestos 1.2

1. Crear una base de datos llamada "blog".
2. Agregar una colección llamada "posts" e insertar 1 documento con una estructura a su elección.
3. Mostrar todas las bases de datos actuales.
4. Eliminar la colección "posts"
5. Eliminar la base de datos "blog" y mostrar las bases de datos existentes.



Problemas propuestos 1.3

1. Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos:

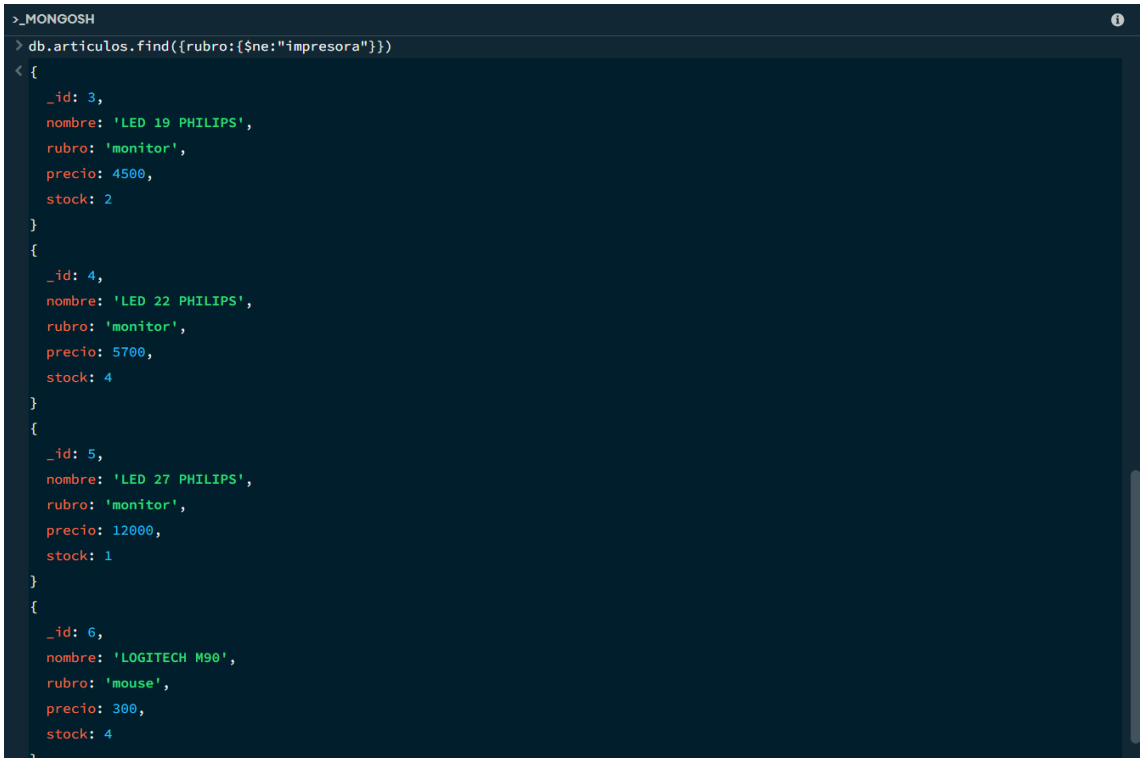
```
use base1
db.articulos.drop()
db.articulos.insertOne(
{
 _id: 1,
 nombre: 'MULTIFUNCION HP DESKJET 2675',
 rubro: 'impresora',
 precio: 3000,
 stock: 20
```

```
 }
)
db.articulos.insertOne(
 {
 _id: 2,
 nombre: 'MULTIFUNCION EPSON EXPRESSION XP241',
 rubro: 'impresora',
 precio: 3700,
 stock: 5
 }
)
db.articulos.insertOne(
 {
 _id: 3,
 nombre: 'LED 19 PHILIPS',
 rubro: 'monitor',
 precio: 4500,
 stock: 2
 }
)
db.articulos.insertOne(
 {
 _id: 4,
 nombre: 'LED 22 PHILIPS',
 rubro: 'monitor',
 precio: 5700,
 stock: 4
 }
)
db.articulos.insertOne(
 {
 _id: 5,
 nombre: 'LED 27 PHILIPS',
 rubro: 'monitor',
 precio: 12000,
 stock: 1
 }
)
db.articulos.insertOne(
 {
 _id: 6,
 nombre: 'LOGITECH M90',
 rubro: 'mouse',
 precio: 300,
 stock: 4
 }
)
```

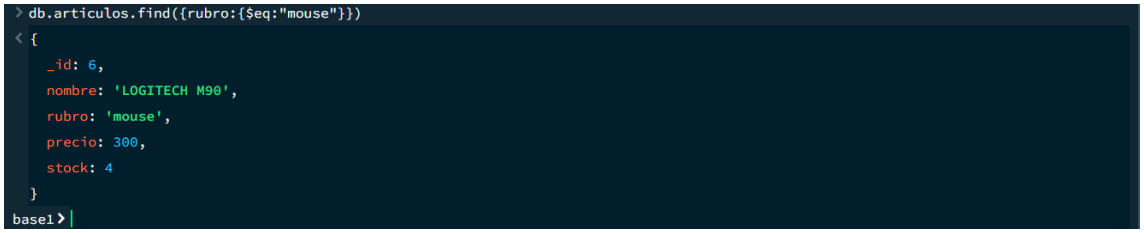
2. Imprimir todos los documentos de la colección 'articulos'.

db.articulos.find()

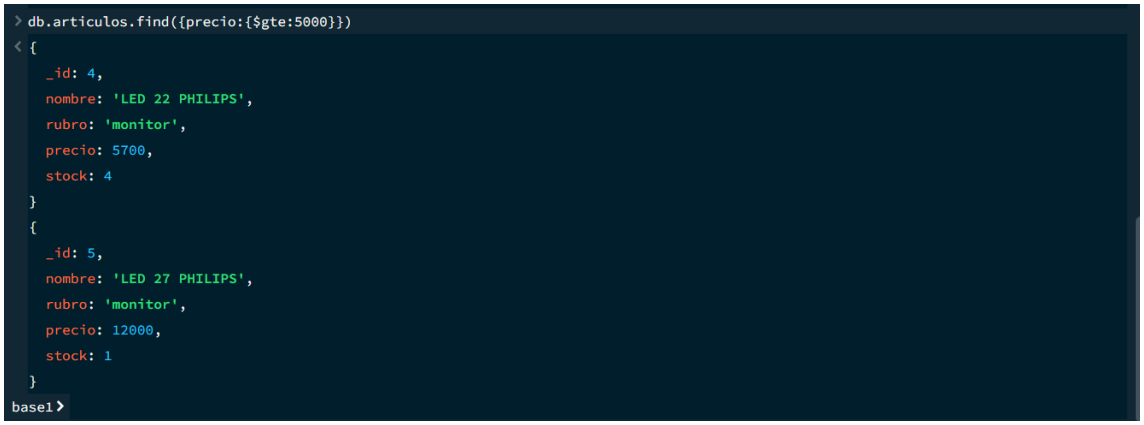
3. Imprimir todos los documentos de la colección 'articulos' que no son impresoras.



4. Imprimir todos los artículos que pertenecen al rubro de 'mouse'.



5. Imprimir todos los artículos con un precio mayor o igual a 5000.



6. Imprimir todas las impresoras que tienen un precio mayor o igual a 3500.

```
> db.articulos.find({rubro:{$eq:"impresora"},precio:{$gte:3500}})
< {
    _id: 2,
    nombre: 'MULTIFUNCION EPSON EXPRESSION XP241',
    rubro: 'impresora',
    precio: 3700,
    stock: 5
  }
base1>
```

7. Imprimir todos los artículos cuyo stock se encuentra comprendido entre 0 y 4.

```
> db.articulos.find({stock:{$gt:0, $lt:4}})
< {
    _id: 3,
    nombre: 'LED 19 PHILIPS',
    rubro: 'monitor',
    precio: 4500,
    stock: 2
  }
  {
    _id: 5,
    nombre: 'LED 27 PHILIPS',
    rubro: 'monitor',
    precio: 12000,
    stock: 1
  }
base1>
```

Problemas propuestos 1.4

1. Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3)
2. Imprimir todos los documentos de la colección 'articulos'.

db.articulos.find()

3. Borrar los documentos de la colección 'articulos' cuyo rubro son impresoras, utilizar las dos sintaxis que permite MongoDB.

```
> db.articulos.deleteMany({rubro:{$eq:"impresora"}})
< {
    acknowledged: true,
    deletedCount: 2
  }
base1>
```

Para la segunda sintaxis vuelvo a insertar las dos impresoras eliminadas

```
> db.articulos.insertOne({
    _id: 1,
    nombre: "MULTIFUNCION HP",
    rubro: "impresora",
    precio: 3000,
    stock: 10
  })
< {
    acknowledged: true,
    insertedId: 1
  }
> db.articulos.insertOne({
    _id: 2,
    nombre: "MULTIFUNCION EPSON EXPRESSION XP241",
    rubro: "impresora",
    precio: 3700,
    stock: 5
  })
< {
    acknowledged: true,
    insertedId: 2
  }
> db.articulos.deleteMany({rubro:"impresora"})
< {
    acknowledged: true,
    deletedCount: 2
  }
base1>
```

4. Borrar todos los artículos que tienen un _id mayor o igual a 5.

```
> db.articulos.deleteMany({_id:{$gte:5}})
< {
    acknowledged: true,
    deletedCount: 2
  }
> db.articulos.find()
< {
    _id: 3,
    nombre: 'LED 19 PHILIPS',
    rubro: 'monitor',
    precio: 4500,
    stock: 2
  }
  {
    _id: 4,
    nombre: 'LED 22 PHILIPS',
    rubro: 'monitor',
    precio: 5700,
    stock: 4
  }
base1>
```

Problemas propuestos (1.5)

1. Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3)
2. Imprimir todos los documentos de la colección 'articulos'.
3. Modificar el precio del mouse 'LOGITECH M90'.

```
> db.articulos.updateOne({nombre:{$eq:"LOGITECH M90"}},{$set:{precio:350}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.articulos.find({nombre:{$eq:"LOGITECH M90"}})
< {
    _id: 6,
    nombre: 'LOGITECH M90',
    rubro: 'mouse',
    precio: 350,
    stock: 4
  }
base1>
```

4. Fijar el stock en 0 del artículo cuyo _id es 6.

```
> db.articulos.updateOne({_id:{$eq:6}},{$set:{stock:0}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.articulos.find({_id:{$eq:6}})
< {
    _id: 6,
    nombre: 'LOGITECH M90',
    rubro: 'mouse',
    precio: 350,
    stock: 0
  }
base1>
```

5. Agregar el campo proveedores con el array ['Martinez','Gutierrez'] para el artículo cuyo _id es 6.

```
> db.articulos.updateOne({_id: {$eq: 6}},{$set: {proveedores: ['Martinez', 'Gutierrez']}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.articulos.find({_id:{$eq:6}})
< {
    _id: 6,
    nombre: 'LOGITECH M90',
    rubro: 'mouse',
    precio: 350,
    stock: 0,
    proveedores: [
      'Martinez',
      'Gutierrez'
    ]
  }
base1>
```

6. Eliminar el campo proveedores para el artículo cuyo _id es 6.

```
> db.articulos.updateOne({_id:{$eq:6}}, {$unset:{proveedores:""}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.articulos.find({_id:{$eq:6}})
< {
    _id: 6,
    nombre: 'LOGITECH M90',
    rubro: 'mouse',
    precio: 350,
    stock: 0
  }
base1>
```

Problemas propuestos (1.6)

1. Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3)
2. Imprimir todos los documentos de la colección 'articulos'.
3. Fijar el stock en cero para todos los artículos del rubro monitor.

```
> db.articulos.updateMany({rubro:{$eq:"monitor"}},{$set:{stock:0}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 3,
    modifiedCount: 3,
    upsertedCount: 0
  }
> db.articulos.find()
< {
    _id: 1,
    nombre: 'MULTIFUNCION HP',
    rubro: 'impresora',
    precio: 3000,
    stock: 10
  }
  {
    _id: 2,
    nombre: 'MULTIFUNCION EPSON EXPRESSION XP241',
    rubro: 'impresora',
    precio: 3700,
    stock: 5
  }
```

```
{
  _id: 3,
  nombre: 'LED 19 PHILIPS',
  rubro: 'monitor',
  precio: 4500,
  stock: 0
}
{
  _id: 4,
  nombre: 'LED 22 PHILIPS',
  rubro: 'monitor',
  precio: 5700,
  stock: 0
}
{
  _id: 5,
  nombre: 'LED 27 PHILIPS',
  rubro: 'monitor',
  precio: 12000,
  stock: 0
}
{
  _id: 6,
  nombre: 'LOGITECH M90',
  rubro: 'mouse',
  precio: 300,
  stock: 4
}
```

4. Agregar un campo llamado 'pedir' con el valor true para todos los artículos que tienen el campo stock en 0.

```
> db.articulos.updateMany({stock:{$eq:0}},{$set:{pedir:true}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 3,
    modifiedCount: 3,
    upsertedCount: 0
  }
```

```
> db.articulos.find({stock:{$eq:0}})
< {
    _id: 3,
    nombre: 'LED 19 PHILIPS',
    rubro: 'monitor',
    precio: 4500,
    stock: 0,
    pedir: true
  }
  {
    _id: 4,
    nombre: 'LED 22 PHILIPS',
    rubro: 'monitor',
    precio: 5700,
    stock: 0,
    pedir: true
  }
  {
    _id: 5,
    nombre: 'LED 27 PHILIPS',
    rubro: 'monitor',
    precio: 12000,
    stock: 0,
    pedir: true
  }
base1 >
```

5. Eliminar el campo 'pedir' de todos los documentos

```
> db.articulos.updateMany({},{$unset:{pedir:""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 3,
  upsertedCount: 0
}
base1>
```

Problemas propuestos (1.7)

1. Crear la colección 'medicamentos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos:
2. Imprimir todos los documentos de la colección 'medicamentos'.
3. Recupere los medicamentos cuyo laboratorio sea 'Roche' y cuyo precio sea menor a 5.

```
> db.medicamentos.find({$and:[{laboratorio:{$eq:"Roche"}},{precio:{$lt:5}}]})
{
  _id: 2,
  nombre: 'Buscapina',
  laboratorio: 'Roche',
  precio: 4.1,
  cantidad: 200
}
base1>
```

4. Recupere los medicamentos cuyo laboratorio sea 'Roche' o cuyo precio sea menor a 5.

```
>_MONGOSH
> db.medicamentos.find({$or:[{laboratorio:{$eq:"Roche"}}, {precio:{$lt:5}}]})
{
  _id: 1,
  nombre: 'Sertal',
  laboratorio: 'Roche',
  precio: 5.2,
  cantidad: 100
}
{
  _id: 2,
  nombre: 'Buscapina',
  laboratorio: 'Roche',
  precio: 4.1,
  cantidad: 200
}
{
  _id: 4,
  nombre: 'Paracetamol 500',
  laboratorio: 'Bago',
  precio: 1.9,
  cantidad: 200
}
{
  _id: 5,
  nombre: 'Bayaspirina',
  laboratorio: 'Bayer',
  precio: 2.1,
  cantidad: 150
```

5. Muestre todos los medicamentos cuyo laboratorio NO sea "Bayer"

```
> db.medicamentos.find({laboratorio:{$not:{$eq:"Bayer"}}})
< {
    _id: 1,
    nombre: 'Sertal',
    laboratorio: 'Roche',
    precio: 5.2,
    cantidad: 100
  }
  {
    _id: 2,
    nombre: 'Buscapina',
    laboratorio: 'Roche',
    precio: 4.1,
    cantidad: 200
  }
  {
    _id: 4,
    nombre: 'Paracetamol 500',
    laboratorio: 'Bago',
    precio: 1.9,
    cantidad: 200
  }
base1>
```

6. Muestre todos los medicamentos cuyo laboratorio sea "Bayer" y cuya cantidad NO sea=100

```
> db.medicamentos.find({$and:[{laboratorio:{$eq:"Bayer"}}, {cantidad:{$not:{$eq:100}}}]})
< {
    _id: 5,
    nombre: 'Bayaspirina',
    laboratorio: 'Bayer',
    precio: 2.1,
    cantidad: 150
  }
  {
    _id: 6,
    nombre: 'Amoxidal jarabe',
    laboratorio: 'Bayer',
    precio: 5.1,
    cantidad: 50
  }
base1>
```

7. Elimine todos los documentos de la colección medicamentos cuyo laboratorio sea igual a "Bayer" y su precio sea mayor a 10.

```
>_MONGOSH
> db.medicamentos.deleteMany({
    $and: [
      { laboratorio: { $eq: "Bayer" } },
      { precio: { $gt: 10 } }
    ]
  })
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.medicamentos.find()
< {
    _id: 1,
    nombre: 'Sertal',
    laboratorio: 'Roche',
    precio: 5.2,
    cantidad: 100
  }
```

```
>_MONGOSH
{
  _id: 2,
  nombre: 'Buscapina',
  laboratorio: 'Roche',
  precio: 4.1,
  cantidad: 200
}
{
  _id: 4,
  nombre: 'Paracetamol 500',
  laboratorio: 'Bago',
  precio: 1.9,
  cantidad: 200
}
{
  _id: 5,
  nombre: 'Bayaspirina',
  laboratorio: 'Bayer',
  precio: 2.1,
  cantidad: 150
}
{
  _id: 6,
  nombre: 'Amoxidal jarabe',
  laboratorio: 'Bayer',
  precio: 5.1,
  cantidad: 50
}
base1>
```

8. Cambie la cantidad por 200, a todos los medicamentos de "Roche" cuyo precio sea mayor a 5

```
> db.medicamentos.updateMany({$and:[{laboratorio:{$eq:"Roche"}},{precio:{$gt:5}}]},{$set:{cantidad:200}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
> db.medicamentos.find({$and:[{laboratorio:{$eq:"Roche"}},{precio:{$gt:5}}]})
< {
    _id: 1,
    nombre: 'Sertal',
    laboratorio: 'Roche',
    precio: 5.2,
    cantidad: 200
  }
base1>
```

9. Borre los medicamentos cuyo laboratorio sea "Bayer" o cuyo precio sea menor a 3

```
> db.medicamentos.deleteMany({$or:[{laboratorio:{$eq:"Bayer"}}, {precio:{$lt:3}}]})
< {
    acknowledged: true,
    deletedCount: 3
  }
> db.medicamentos.find()
< {
    _id: 1,
    nombre: 'Sertal',
    laboratorio: 'Roche',
    precio: 5.2,
    cantidad: 200
  }
  {
    _id: 2,
    nombre: 'Buscapina',
    laboratorio: 'Roche',
    precio: 4.1,
    cantidad: 200
  }
base1 >
```